

Deployment & Go-Live Guide Notarial SaaS Platform

Hostinger VPS & Multi-Office Onboarding — Step-by-step procedure, prerequisites, exact commands, checks and error handling.

Reference document for the Legal Notary founder and technical team (Dev, Support, Sales) covering installation, production go-live, and day-to-day operation of the platform.

Table of Contents

Phase 0 — Toolkit & prerequisites

Phase 1 — Preparing the Hostinger VPS server

Phase 2 — Deploying the SaaS code on the VPS

Phase 3 — Domain name, Nginx reverse proxy & SSL

Phase 4 — Office onboarding guide (4 real-world cases)

Phase 5 — Maintenance checklist & day-to-day management

PRO TIP — How to read this document

Each phase follows the logic of an assembly manual: numbered steps, commands to copy-paste exactly as shown (only replace the values in **UPPERCASE**), and a validation test at the end of each block. Never move on to the next step until the validation test is green.

PHASE 0

Toolkit & prerequisites

Everything you need on hand before opening the terminal.

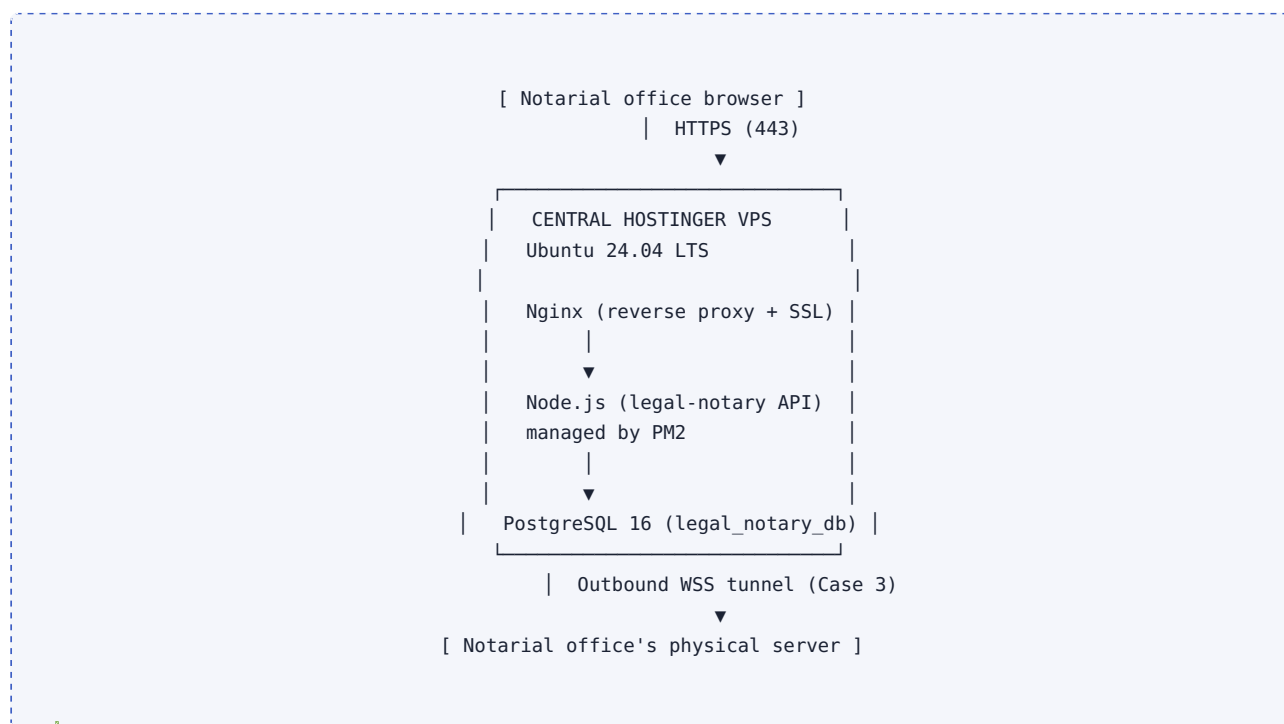
0.1 — Supply list

Item	Detail / Reference	Where to get it
VPS server	Hostinger VPS KVM 2 (2 vCPU / 8 GB RAM) to start, KVM 4 if > 15 active offices — OS Ubuntu 24.04 LTS	hpanel.hostinger.com → VPS → New VPS
Domain name	Primary domain, e.g. <code>notaires.ci</code> or <code>legalnotary.ci</code> (each office's subdomain is generated automatically: <code>[office-code].notaires.ci</code>)	Hostinger Domains or local registrar (NIC.ci)
SSH access	Terminal (macOS/Linux, built in), or PuTTY + WinSCP on Windows	putty.org
SSH key	Key pair generated locally (<code>ssh-keygen</code>) — avoids password-based root access	Generated locally
Git account	Access to the <code>legal-notary</code> repository (GitHub/GitLab, <code>main</code> branch = production)	github.com
Transactional mailbox	An SMTP provider (e.g. Brevo, Resend, or Hostinger SMTP) to send credentials to offices	Depends on provider chosen

WARNING

Never configure the production environment to be used as root for everyday work. Root is only used to create a dedicated system user (Step 1.1). All routine work afterward must be done as that user.

0.2 — Architecture overview



The Hostinger server is the heart of the system: it hosts the API, the database, and serves the subdomains of every office running in Cloud mode. Offices running On-Premise or Hybrid mode (Case 3, Phase 4) connect to it through an encrypted outbound tunnel, without ever opening an inbound port at the client site.

VALIDATION TEST — Phase 0

Before continuing, make sure you have: (1) Hostinger panel credentials, (2) the chosen domain name pointed at a registrar, (3) an SSH key pair generated locally, (4) access to the Git repository. If all 4 boxes are checked, move on to Phase 1.

PHASE 1

Preparing the Hostinger VPS server

Step-by-step manual — from the first connection to a database ready for use.

Step 1.1 — First SSH connection

1.1

Connect to the VPS for the first time

From your local terminal:

```
# Replace YOUR_HOSTINGER_IP with the IP given in hPanel
ssh root@YOUR_HOSTINGER_IP
```

Accept the security fingerprint (`yes`) then enter the root password sent by email when the VPS was created.

Immediately create a dedicated system user (never work as root day to day):

```
adduser legalnotary
usermod -aG sudo legalnotary
rsync --archive --chown=legalnotary:legalnotary ~/.ssh /home/legalnotary
```

PRO TIP

Log out and reconnect immediately with `ssh legalnotary@YOUR_HOSTINGER_IP` to confirm access works BEFORE disabling password-based root login.

Step 1.2 — System update & required packages

1.2

Update Ubuntu and install the base tools

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git nginx certbot python3-certbot-nginx ufw build-essential
```

Install Node.js 20 LTS via NodeSource:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v # should print v20.x.x
```

Install PostgreSQL 16:

```
sudo apt install -y postgresql-16 postgresql-contrib
sudo systemctl enable postgresql --now
```

Install PM2 (Node.js process manager) globally:

```
sudo npm install -g pm2
```

Step 1.3 — UFW firewall

1.3

Open only the 3 strictly necessary ports

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
```

WARNING

Never run `ufw enable` before allowing the SSH port (`OpenSSH`). A mistake here locks you out of the server and requires reinstalling or using the Hostinger rescue console (hPanel → VPS → Browser console).

Step 1.4 — PostgreSQL configuration

1.4

Create the database, the application user, and the encryption extension

```
sudo -u postgres psql
```

Inside the `psql` console that opens:

```
CREATE DATABASE legal_notary_db;  
CREATE USER notaire_user WITH ENCRYPTED PASSWORD 'REPLACE_WITH_A_STRONG_PASSWORD';  
GRANT ALL PRIVILEGES ON DATABASE legal_notary_db TO notaire_user;  
\c legal_notary_db  
CREATE EXTENSION IF NOT EXISTS pgcrypto;  
\q
```

PRO TIP

Generate the password with `openssl rand -base64 24` and store it right away in your team's password manager — you'll reuse it in Step 2.2 inside the `.env` file.

VALIDATION TEST — Phase 1

Run `sudo -u postgres psql -c "\l" : legal_notary_db` should appear in the list. Run `sudo ufw status` : only 22, 80, and 443 should show `ALLOW` . If both are correct, move on to Phase 2.

PHASE 2

Deploying the SaaS code on the VPS

From the Git clone to a process that keeps running.

Step 2.1 — Cloning the project

2.1

```
sudo mkdir -p /var/www/legal-notary
sudo chown -R legalnotary:legalnotary /var/www/legal-notary
cd /var/www
git clone https://github.com/YOUR-ORG/legal-notary.git
cd legal-notary
git checkout main
```

Step 2.2 — Production environment file

2.2

Create the `.env` file — it must **never** be committed to Git:

```
cd /var/www/legal-notary/legal-notary-server
nano .env
```

Content to adapt with your real values:

```
NODE_ENV=production
PORT=4000
DATABASE_URL=postgresql://notaire_user:YOUR_PASSWORD@localhost:5432/legal_notary_db
JWT_SECRET=value generated with: openssl rand -hex 32
CORS_ORIGIN=https://notaires.ci,https://*.notaires.ci
SMTP_HOST=...
SMTP_USER=...
SMTP_PASSWORD=...
```

WARNING

Add `.env` to `.gitignore` before the first commit on this server. Any secret pushed to GitHub, even by mistake, must be treated as compromised and rotated immediately (JWT_SECRET, DB passwords, SMTP keys).

Step 2.3 — SQL migrations & internal accounts

2.3

```
cd /var/www/legal-notary/legal-notary-server
npm install --omit=dev
npm run migrate
npm run seed:admin # creates the publisher's SuperAdmin account (Phase 5)
```

Step 2.4 — PM2, the process guardian

2.4

Start the API and keep it alive through any VPS reboot

```
pm2 start src/server.js --name legal-notary-api
pm2 startup # copy and run the sudo command it prints
pm2 save
```

Useful day-to-day commands:

```
pm2 status # check whether the API is running
pm2 logs legal-notary-api # follow live logs
pm2 restart legal-notary-api # after a code update
```

VALIDATION TEST — Phase 2

Run `curl http://127.0.0.1:4000/health` (or your API's health endpoint): you should get a `200 OK` response. Run `pm2 status`: the `legal-notary-api` process should be `online`. Reboot the VPS (`sudo reboot`) and reconnect: PM2 should have restarted the API on its own.

PHASE 3

Domain name, Nginx reverse proxy & SSL

Making the application reachable over HTTPS on a real domain name.

Step 3.1 — DNS zone on Hostinger

3.1

In hPanel → Domains → DNS / Zone Editor, add:

Type	Name	Value	TTL
A	@	VPS IP	14400
A	* (wildcard)	VPS IP	14400
A	www	VPS IP	14400

The wildcard `*` record is what lets every office get its `[office-code].notaires.ci` subdomain automatically, with no extra DNS configuration.

Step 3.2 — Nginx configuration (reverse proxy)

3.2

```
sudo nano /etc/nginx/sites-available/legal-notary
```

```
server {
    listen 80;
    server_name notaires.ci *.notaires.ci;

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    add_header Strict-Transport-Security "max-age=63072000" always;
}
```

Enable the site, then test the syntax before reloading:

```
sudo ln -s /etc/nginx/sites-available/legal-notary /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Step 3.3 — Let's Encrypt SSL certificate (wildcard)

3.3

```
sudo certbot --nginx -d notaires.ci -d *.notaires.ci
```

Certbot will ask for DNS-01 validation for the wildcard (HTTP-01 is not possible): follow the on-screen instructions to add the temporary TXT record it requests in the Hostinger DNS zone.

Automatic renewal is already scheduled by certbot; verify it:

```
sudo certbot renew --dry-run
```

WARNING

A Let's Encrypt wildcard certificate requires DNS-01 validation, not the automatic HTTP-01 flow. Make sure you have access to the DNS zone when you run this command — the process is interactive and expires after a few minutes.

VALIDATION TEST — Phase 3

Open <https://notaires.ci> and <https://test123.notaires.ci> in a browser: the HTTPS padlock should appear with no warning on both. The platform is now live.

PHASE 4

Office onboarding & go-live guide

The 4 procedure sheets for the sales & support team — one case, one sheet, followed from the first to the last point.

Case 1 — The office has no equipment

100% turnkey Cloud

1. In the SuperAdmin console, create a new office and assign it a unique `office-code` (e.g. `kouame-abj`).
2. The system automatically provisions `https://kouame-abj.notaires.ci` (the Phase 3 wildcard SSL already covers this subdomain — no manual action needed).
3. Generate credentials for the lead notary (email + temporary password to be changed on first login).
4. Send the welcome kit (link, credentials, quick-start PDF guide) via the transactional email service.
5. Schedule a 20-minute onboarding call.

Case 2 — The office has its own domain name

Custom domain

Example: the office already owns `notaire-kouame.ci` and wants to use it instead of the generic subdomain.

1. Ask the office to add a `CNAME` record in its own DNS zone: `notaire-kouame.ci` → `kouame-abj.notaires.ci` (or an `A` record to the VPS IP if its registrar doesn't support `CNAME` on the root — use `www` in that case).
2. Add the custom domain in the SuperAdmin console ("Linked domains" module).
3. On the VPS, add the domain to Nginx's `server_name` and run `sudo certbot --nginx -d notaire-kouame.ci` to issue a dedicated SSL certificate.
4. Check DNS propagation (`dig notaire-kouame.ci`) before sharing the new link with the office.

Case 3 — The office has a physical server on-site

On-Premise / Hybrid

Two sub-modes: **Mode A (On-Premise)** — data stays entirely at the office; **Mode C (Hybrid)** — local processing, with encrypted backup replicated to the central VPS.

1. In the SuperAdmin console, generate a single-use pairing token in the format `PAIR-ETD-xxx`.
2. On the office's machine/server, run the one-line installation command (agent + tunnel):

```
curl -fsSL https://install.notaires.ci/agent.sh | sudo bash -s -- PAIR-ETD-xxx
```

3. The agent establishes an **encrypted outbound tunnel (WSS)** to the central VPS: no inbound port needs to be opened on the office's internet router, avoiding any network configuration on the client side.
4. Check in the SuperAdmin console that the office's status turns to `Connected` (green dot) within 60 seconds of installation.
5. Document in the office record: server model, OS, agent version, installation date.

WARNING

Never ask an office to open an inbound port on its router or firewall for this installation. If a local technician suggests it, that's a sign the installation command wasn't used correctly — restart the standard procedure.

Case 4 — The office has its own cloud storage

BYOS — Bring Your Own Storage

1. Collect the office's dedicated S3 bucket or MinIO instance credentials (endpoint, access key, secret key, bucket name).
2. In the SuperAdmin console, "Storage" module, select `BYOS` and enter these parameters for that office.
3. Run the built-in connection test (write + read of a test file) before confirming.
4. Enable client-side encryption at rest if the storage provider doesn't already do it natively.

PHASE 5

Maintenance checklist & day-to-day management

What the founder should be able to do alone, without relying on a developer, once the platform is live.

5.1 — Accessing the SuperAdmin console from anywhere

- ☐ Login URL: `https://admin.notaires.ci` (create as a dedicated subdomain, separate from the offices).
- ☐ Enable two-factor authentication (2FA) on the SuperAdmin account — non-negotiable given the sensitivity of notarial data.
- ☐ Restrict admin access to a trusted IP list where possible, on top of 2FA.

5.2 — Checking backup health (3-2-1 WORM)

The 3-2-1 principle: 3 copies of the data, on 2 different media, with 1 off-site. WORM (Write Once Read Many) means that even a compromised admin account cannot alter or delete a backup once it has been written.

- ☐ Check weekly, in the SuperAdmin console, that the latest backup job for the `legal_notary_db` database is marked `Success` and dated less than 24 hours ago.
- ☐ Check that the off-site backup (immutable bucket) is actually receiving files (size > 0, recent date).
- ☐ Run a full restore test once a quarter on a test environment — a backup that has never been restored is not a reliable backup.

5.3 — Managing the internal team & the permissions matrix

In the SuperAdmin console → Team:

1. Add the team member by email (Dev, Sales, Support, or Assistant) — an activation link is sent automatically.
2. Check the granted rights in the permissions matrix (view offices, manage billing, technical support access, server configuration access).
3. Review this matrix whenever a role changes, and revoke access immediately when a team member leaves.

5.4 — Emergency contacts & disaster recovery failover

The Disaster Recovery Plan (DRP) lets you switch to a backup server in case of a major failure on the primary VPS.

1. Keep an "Emergency contacts" sheet up to date: hosting provider (Hostinger support), on-call developer, network provider.
2. The backup server must hold a recent copy of the database and code, ready to be activated.
3. In case of an outage: switch DNS (A record) to the backup server's IP; with a low TTL already configured, propagation takes minutes, not hours.
4. Document every incident afterward (cause, duration, actions taken) to shorten the failover time next time.

WARNING — A commonly overlooked point: production error reporting

Once live, your platform may run not only on this central Hostinger VPS, but also — via Phase 4's Case 3 — on physical servers located directly at notarial offices. Without a dedicated tool, an application error occurring on a client's server reaches no one — not you, not your technical team. Before onboarding your first offices in production, put in place:

- A centralized error-tracking tool such as **Sentry** (or an equivalent), integrated into the Node.js API and, where possible, into the Case 3 agent.
- Retained application logs (`pm2 logs` alone isn't enough over time — consider `pm2-logrotate` plus export to centralized storage).
- An automatic alert (email or Slack) triggered on critical errors, rather than relying on a client to report the problem.

This matters even more because the server involved isn't always yours: without automatic reporting, an outage at an On-Premise office can stay invisible for days.

VALIDATION TEST — Phase 5

You can answer without hesitation the following 4 questions: where do you check whether the last backup succeeded? Who do you contact if the VPS goes down? How do you revoke access for a team member who leaves? Where would you be alerted if an On-Premise office hit a blocking error? If any answer is missing, that's the one to fix before your first production client.

Document written for Legal Notary (BETHEL TIME) — update whenever the architecture changes significantly (hosting provider change, new onboarding mode, database migration).